
Table of Contents

PREAMBLE	1
=====	=====
1. COMMENTING	1
=====	=====
=====	=====
2. THINKING IN THREE DOMAINS	1
=====	=====
=====	=====
3. THINKING IN THREE DOMAINS	12
=====	=====
=====	=====
ALL FUNCTIONS SUPPORTING THIS CODE %% function pzplot(b,a)	21

PREAMBLE

DO NOT REMOVE THE LINE BELOW

```
clear; close all;
```

=====

=====

1. COMMENTING

=====

=====

NOTHING TO ADD HERE

=====

=====

2. THINKING IN THREE DOMAINS

=====

=====

```
% MAKE SURE FILES BELOW ARE IN SAME DIRECTORY AS THIS FILE  
[x, fs] = audioread('music.wav');
```

```

% DEFINE TIME AND FREQUENCY AXES
t = (0:length(x)-1)/fs;
w = -pi:pi/4000:pi-pi/4000;

% -----
% 2(a)
% -----
% What type of filter did you design: low-pass, high-pass,
% band-pass, band-stop, all-pass? Why?

% I designed a low-pass filter because the string bass occupies
% lower frequencies while the trumpet occupies higher frequencies.
% A low-pass filter allows me to retain the bass while removing
% the trumpet.

% -----
% 2(b)
% -----
% Did you create an FIR or IIR filter? Why?

% I chose to implement an IIR filter because it can achieve useful
% low-pass behavior with fewer coefficients than an FIR filter.
% This helps me satisfy the requirement of having no more than 10
% non-zero coefficients.

% DESIGN A STRONGER CAUSAL IIR LOW-PASS FILTER (NO TOOLBOX)

% Four zeros at  $z = -1$  to strongly attenuate high frequencies
myzeros = [-1 -1 -1 -1];

% Four poles near  $z = 1$  to preserve low frequencies
mypoles = [0.7 0.7 0.7 0.7];

% Convert poles and zeros to filter coefficients
[b2, a2] = pz2ba(mypoles, myzeros);

% Normalize gain so that  $|H(0)| = 1$ 
G = abs(sum(a2) / sum(b2));
b2 = G * b2;

% FILTER SIGNAL
y = filter(b2, a2, x);

% -----
% 2(c)
% -----

```

```

% (c) Plot the frequency-domain magnitudes and the time-domains for the
input and output signals
% ( $|X(\omega)|$  and  $|Y(\omega)|$ ) (use Question #1 as guidance).

% COMPUTE FREQUENCY-DOMAIN REPRESENTATIONS
X = DTFT(x, w);
Y = DTFT(y, w);

% PLOT INPUT/OUTPUT FREQUENCY-DOMAIN MAGNITUDES
figure
subplot(2,1,1)
plot(w, abs(X))
xlabel('Normalized Frequency (Radians)')
ylabel('|X(\omega)|')
title('Q2: Input Signal Spectrum |X(\omega)|')

subplot(2,1,2)
plot(w, abs(Y))
xlabel('Normalized Frequency (Radians)')
ylabel('|Y(\omega)|')
title('Q2: Output Signal Spectrum |Y(\omega)| (Low-Pass Filter)')

snapnow;

% PLOT INPUT/OUTPUT TIME-DOMAIN SIGNALS
figure
subplot(2,1,1)
plot(t, x)
xlabel('Time (s)')
ylabel('Amplitude')
title('Q2: Input Signal x(t)')

subplot(2,1,2)
plot(t, y)
xlabel('Time (s)')
ylabel('Amplitude')
title('Q2: Filtered Output Signal y(t) (Low-Pass)')

snapnow;

% -----
% 2(d)
% -----
% What do these plots tell should you about your filter?

% From these plots, I can see that my filter is working as intended.
% The output spectrum retains the strong low-frequency components near
%  $\omega = 0$  while significantly reducing the higher-frequency content.
% This shows that the bass is being preserved while the trumpet is
% being attenuated. In the time domain, the output still resembles the
% input, but the amplitudes are shaped slightly different
% because much of the high-frequency content has been removed.

```

```

% -----
% 2(e)
% -----
% Plot the filter impulse response h[n].

Nimp = 120;
delta = [1; zeros(Nimp-1,1)];
h2 = filter(b2, a2, delta);
n = 0:Nimp-1;

figure
stem(n, h2, 'filled')
xlabel('Samples')
ylabel('h[n]')
title('Q2: Impulse Response h_2[n] (IIR Low-Pass Filter)')

snapnow;

% -----
% 2(f)
% -----
% What does the impulse response tell you about your filter?

% The impulse response shows that my filter is causal because the
% response begins at n = 0. The response gradually decays over time,
% which indicates that the filter is stable. Since the impulse
% response does not terminate after a finite number of samples, I
% can confirm that this is an IIR filter.

% -----
% 2(g)
% -----
% Plot the filter magnitude response |H(w)|.

% COMPUTE FILTER FREQUENCY RESPONSE
H2 = DTFT(h2, w);

figure
plot(w, abs(H2))
xlabel('Normalized Frequency (Radians)')
ylabel('|H(\omega)|')
title('Q2: Magnitude Response |H_2(\omega)| (IIR Low-Pass)')

snapnow;

% -----
% 2(h)
% -----
% What does the frequency response tell you about your filter?

% From the magnitude of the frequency response, I can see that my

```

```

% filter behaves like a low-pass filter. Frequencies near omega = 0
% are passed with high gain, while higher frequencies are strongly
% reduced. This confirms that the filter keeps the bass while
% reducing the trumpet.

% -----
% 2(i)
% -----
% Plot the pole-zero plot of your filter.
figure
pzplot(b2, a2)
axis equal
title('Q2: Pole-Zero Plot (IIR Low-Pass Filter)')

snapnow;
% -----
% 2(j)
% -----
% What does the pole-zero plot tell you about your filter?

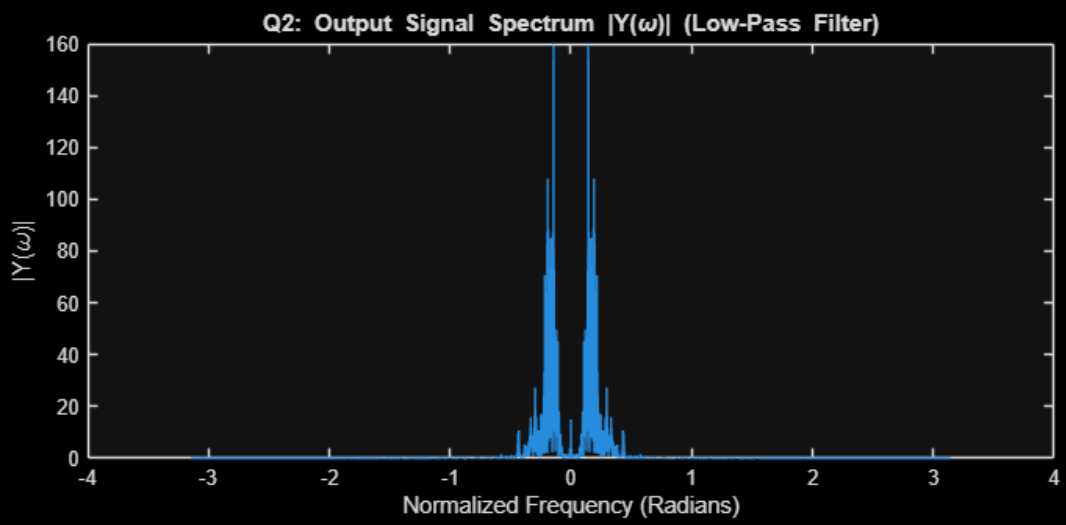
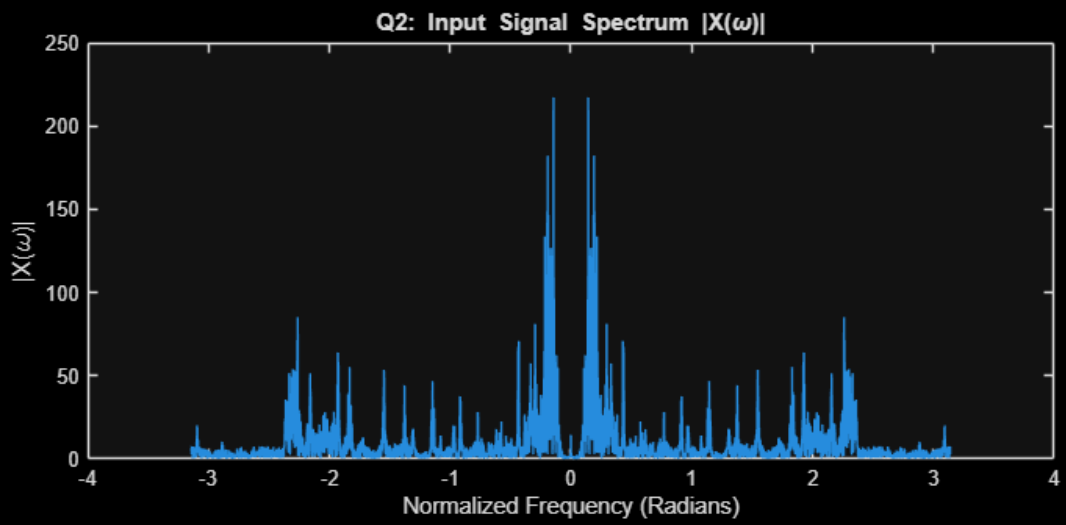
% The pole-zero plot shows that my filter has repeated zeros at
% z = -1, which strongly attenuate high-frequency components, and
% repeated poles at z = 0.7, which reinforce low-frequency
% components. Since the poles lie inside the unit circle, the filter
% is stable.

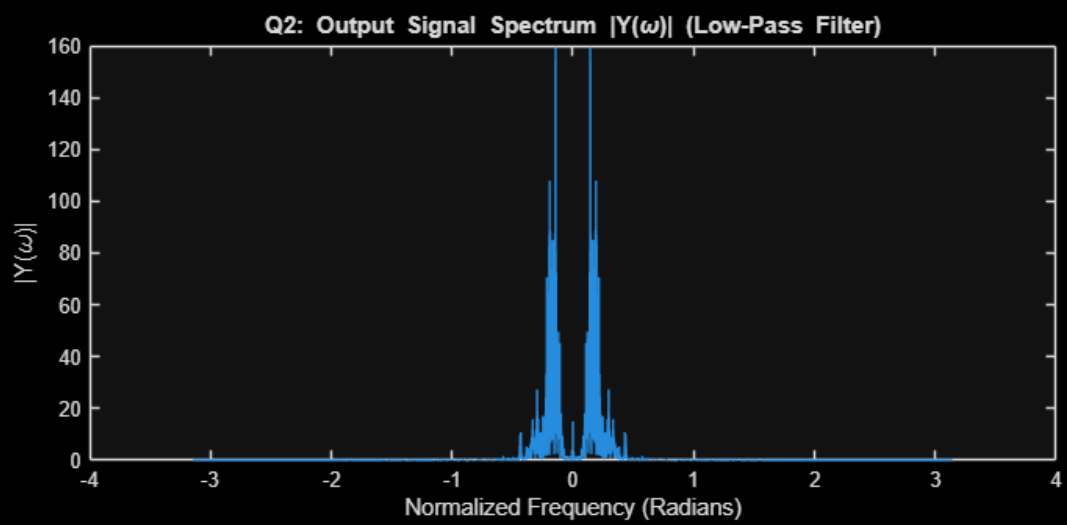
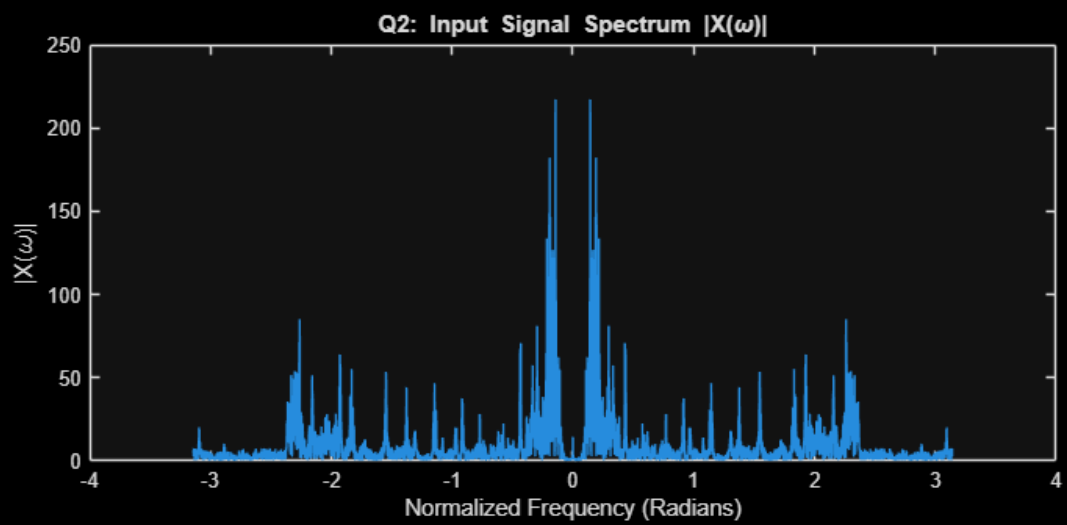
% -----
% 2(k)
% -----
% Save (as q2.wav) your filtered audio
y2 = y./max(abs(y));
audiowrite('q2.wav', y2, fs);

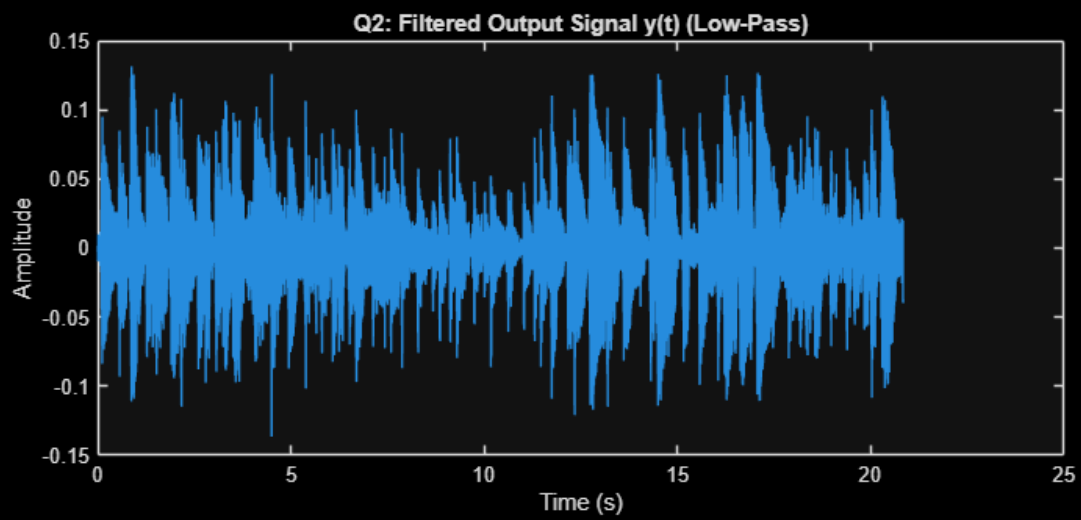
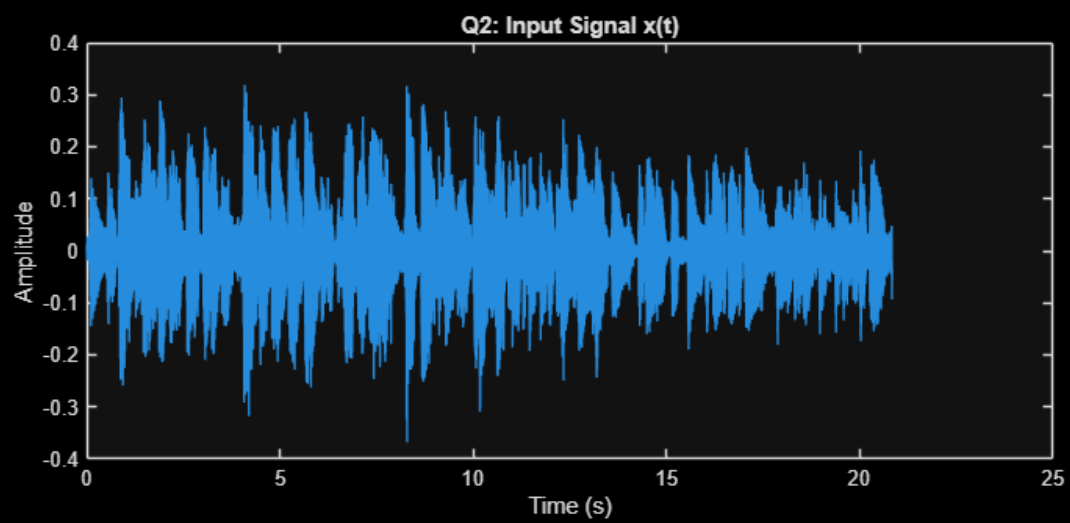
% -----
% 2(l)
% -----
% Save (as q2.mat) your filter coefficients using MATLAB's save function.
The filter coefficients
% should be in variables called b2 (numerator) and a2 (denominator).

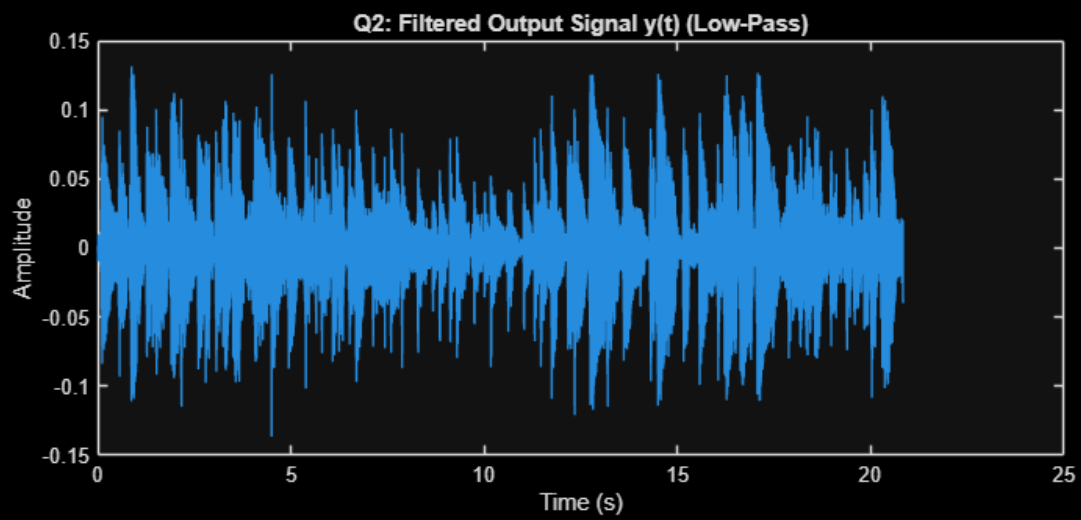
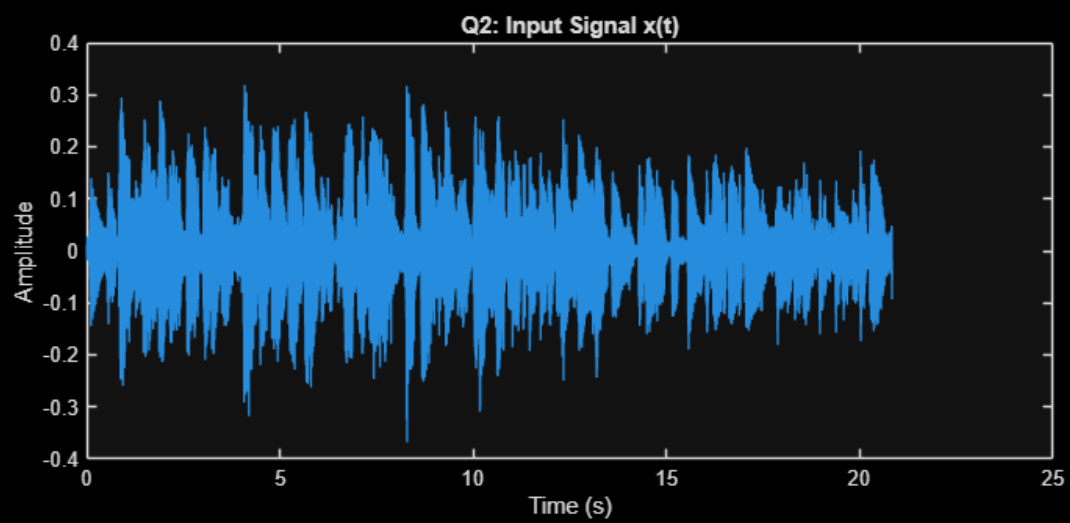
save('q2.mat', 'b2', 'a2');

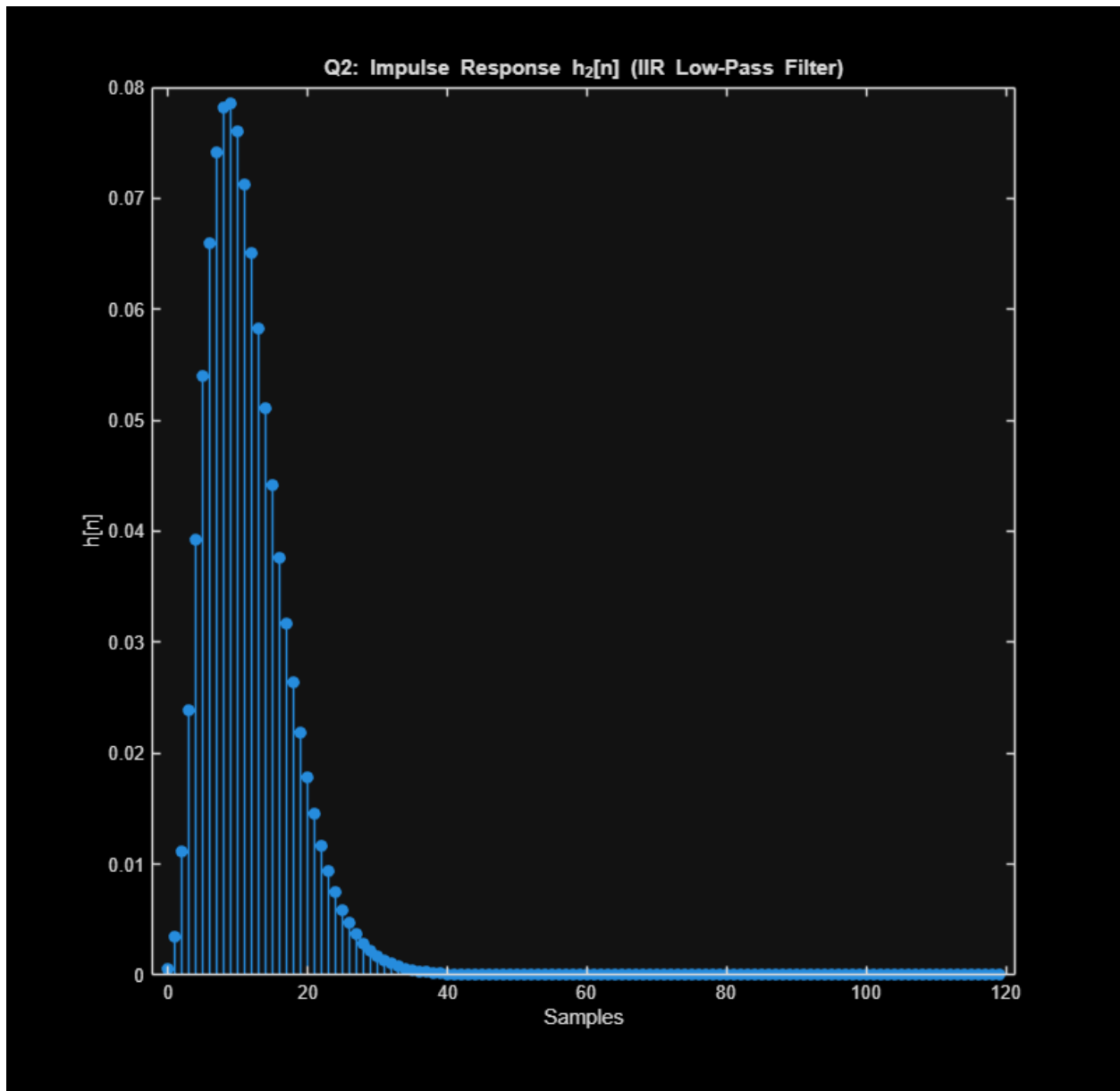
```

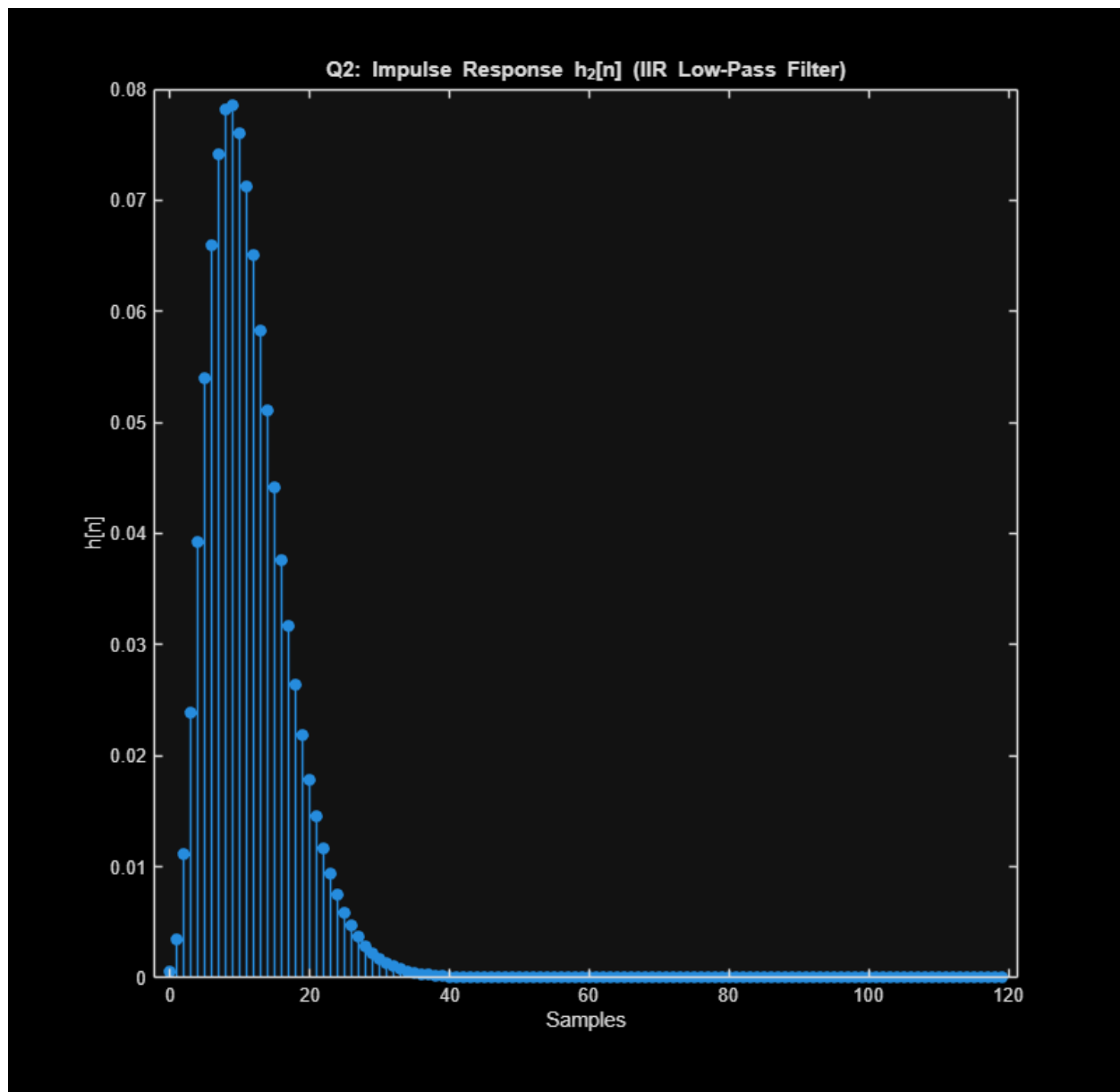


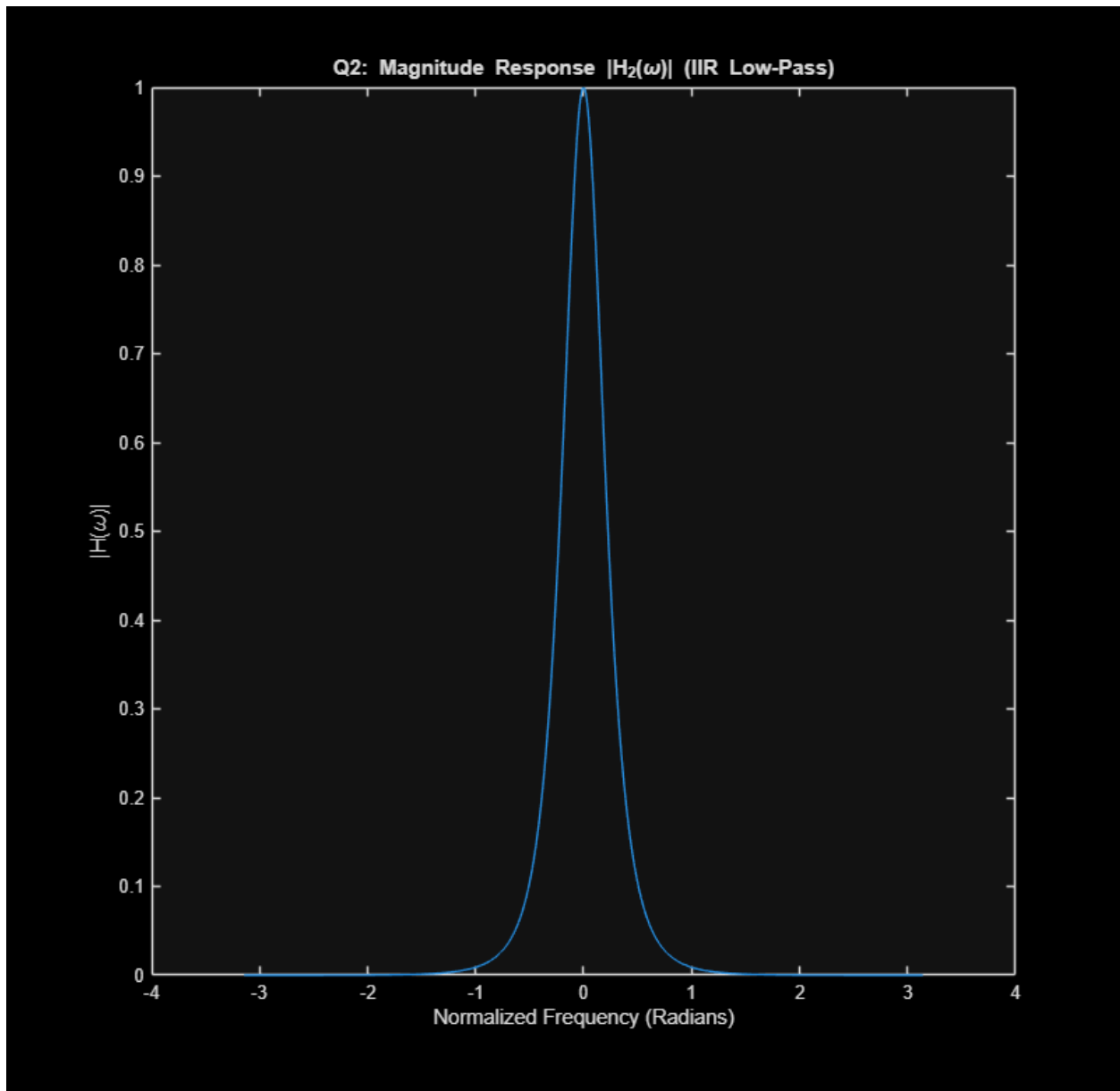












=====

=====

3. THINKING IN THREE DOMAINS

=====

=====

```
% MAKE SURE FILES BELOW ARE IN SAME DIRECTORY AS THIS FILE  
[x, fs] = audioread('music.wav');
```

```

% -----
% 3(a)
% What type of filter did you design: low-pass, high-pass,
% band-pass, band-stop, all-pass? Why?
% -----

% I designed a high-pass filter because the trumpet occupies higher
% frequencies while the string bass occupies lower frequencies.
% A high-pass filter allows me to retain the trumpet while reducing
% the string bass.

% -----
% 3(b)
% Did you create an FIR or IIR filter? Why?
% -----

% I created an FIR filter because Question 3 specifically requires
% the filter to be FIR. I designed the filter by cascading multiple
%  $H_b(z)$  sections, which places zeros across a range of low
% frequencies. This creates a wide stopband that removes the bass
% while preserving the higher-frequency trumpet content.

% DESIGN FIR HIGH-PASS USING MULTIPLE NOTCHES (COURSE STYLE)

b3 = 1;
a3 = 1;

% Place zeros across low-frequency region
w_vals = 0.2:0.1:1.0;

for k = 1:length(w_vals)
    w0 = w_vals(k);
    b_k = conv([1 -exp(1j*w0)], [1 -exp(-1j*w0)]);
    b3 = conv(b3, real(b_k));
end

% Normalize at omega = pi
n_hp = 0:length(b3)-1;
Hpi = sum(b3 .* exp(-1j*pi*n_hp));
b3 = b3 / abs(Hpi);

% FILTER SIGNAL
y = filter(b3, a3, x);

% -----
% 3(c)

% Plot the frequency-domain magnitudes and the time-domains for the input
% and output signals

```

```

% (|X(ω)| and |Y(ω)|) (use Question #1 as guidance).
% -----

% DEFINE TIME AND FREQUENCY AXES
t = (0:length(x)-1)/fs;
w = -pi:pi/4000:pi-pi/4000;

% COMPUTE FREQUENCY-DOMAIN REPRESENTATIONS
X = DTFT(x, w);
Y = DTFT(y, w);

figure
subplot(2,1,1)
plot(w, abs(X))
xlabel('Normalized Frequency (Radians)')
ylabel('|X(\\omega)|')
title('Q3: Input Signal Spectrum |X(\\omega)|')

subplot(2,1,2)
plot(w, abs(Y))
xlabel('Normalized Frequency (Radians)')
ylabel('|Y(\\omega)|')
title('Q3: Output Signal Spectrum |Y(\\omega)| (High-Pass Filter)')

snapnow;

figure
subplot(2,1,1)
plot(t, x)
xlabel('Time (s)')
ylabel('Amplitude')
title('Q3: Input Signal x(t)')

subplot(2,1,2)
plot(t, y)
xlabel('Time (s)')
ylabel('Amplitude')
title('Q3: Filtered Output Signal y(t) (High-Pass)')

snapnow;

% -----
% 3(d) What do these plots tell you about your filter?
% -----
% From these plots, I can see that my filter is working as a
% high-pass filter. The output spectrum shows that much of the
% low-frequency content has been reduced, while the higher-
% frequency trumpet content remains. In the time domain, the output
% signal has reduced low-frequency energy and sounds thinner because
% much of the bass has been removed.

% -----
% 3(e) Plot the filter impulse response h[n]

```

```

% -----

imp = [1 zeros(1, length(b3)-1)];
h = filter(b3, a3, imp);

figure
stem(0:length(h)-1, h, 'filled')
xlabel('Samples')
ylabel('h[n]')
title('Q3: Impulse Response h_3[n] (FIR High-Pass Filter)')

snapnow;

% -----
% 3(f) What does the impulse response tell you about your filter?
% -----
% The impulse response shows that my filter is FIR because it has a
% finite number of nonzero samples and then becomes zero. It is also
% causal because the response begins at n = 0. The symmetric pattern
% of the coefficients is consistent with the structure of the FIR
% filter I designed.

% -----
% 3(g) Plot the filter magnitude response |H(w)|.
% -----

w = -pi:pi/4000:pi-pi/4000;
H = DTFT(b3, w); % since FIR, a3 = 1

figure
plot(w, abs(H))
xlabel('Normalized Frequency (Radians)')
ylabel('|H(\omega)|')
title('Q3: Magnitude Response |H_3(\omega)| (FIR High-Pass)')

snapnow;

% -----
% 3(h) What does the frequency response tell you about your filter?
% -----
% From the magnitude of the frequency response, I can see that my
% filter behaves like a high-pass filter. Frequencies near omega = 0
% are strongly weakened, while frequencies closer to plus or minus
% pi are passed with much larger gain. This confirms that the filter
% removes the low-frequency bass and retains the higher-frequency
% trumpet content.

% -----
% 3(i) Plot the pole-zero plot of your filter
% -----

figure

```

```

pzplot(b3, a3)
title('Q3: Pole-Zero Plot (FIR High-Pass Filter)')
axis equal

snapnow;

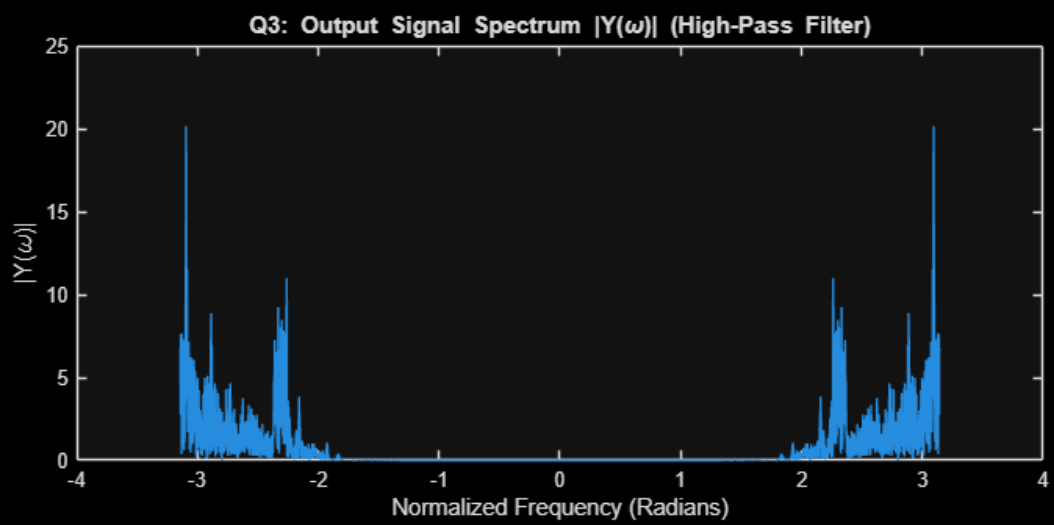
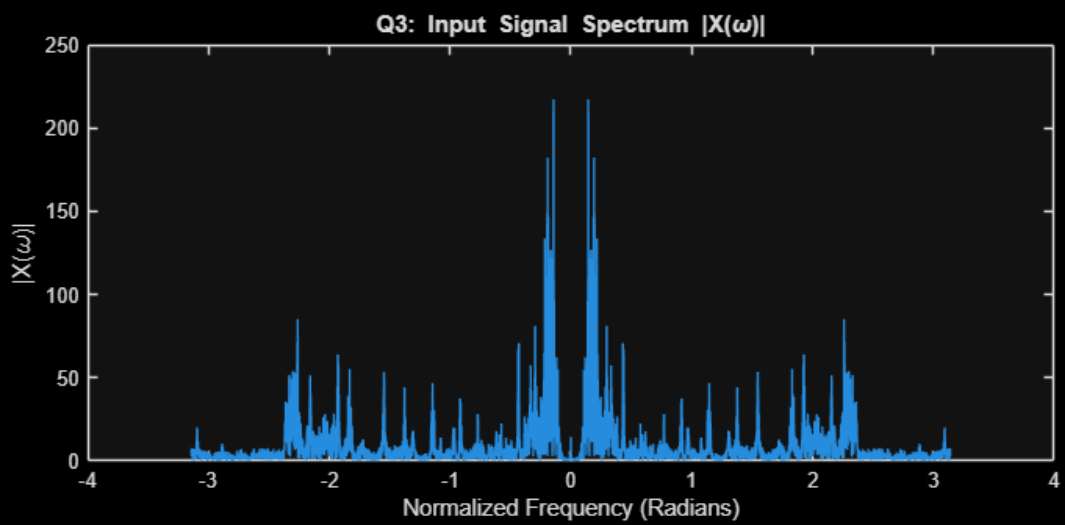
% -----
% 3(j) What does the pole-zero plot tell you about your filter?
% -----
% The pole-zero plot shows that the filter has multiple zeros placed
% along the unit circle at low frequencies. These zeros create strong
% reduction in the low-frequency region, which removes the bass
% components of the signal. Since this is an FIR filter, there are no
% poles except the one at the origin (no feedback poles). The placement of
% these zeros
% explains why the filter behaves as a high-pass filter, allowing the
% higher-frequency trumpet content to pass while reducing the bass.

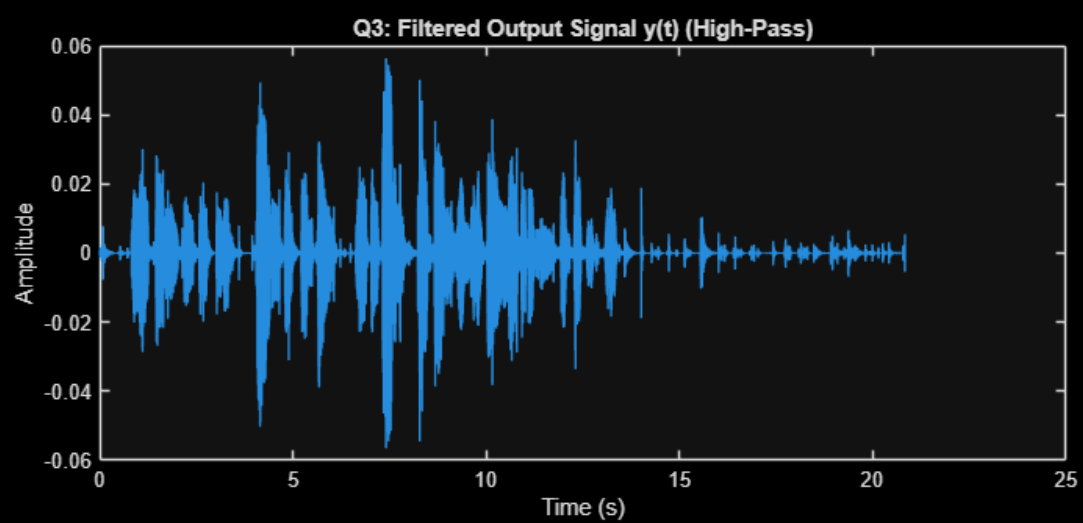
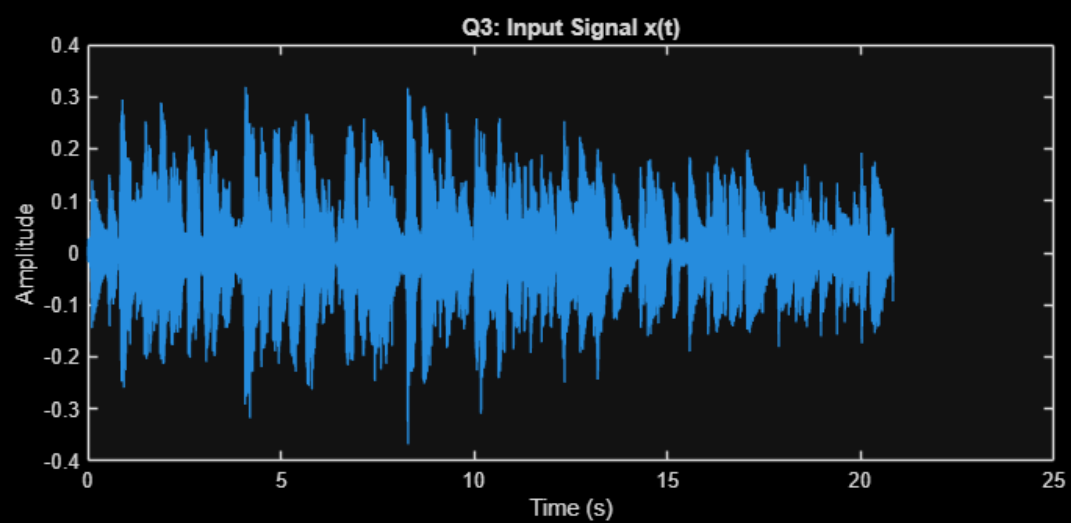
% -----
% 3(k) Save (as q3.wav) your filtered audio
% -----
y3 = y ./ max(abs(y)); % normalize
audiowrite('q3.wav', y3, fs);

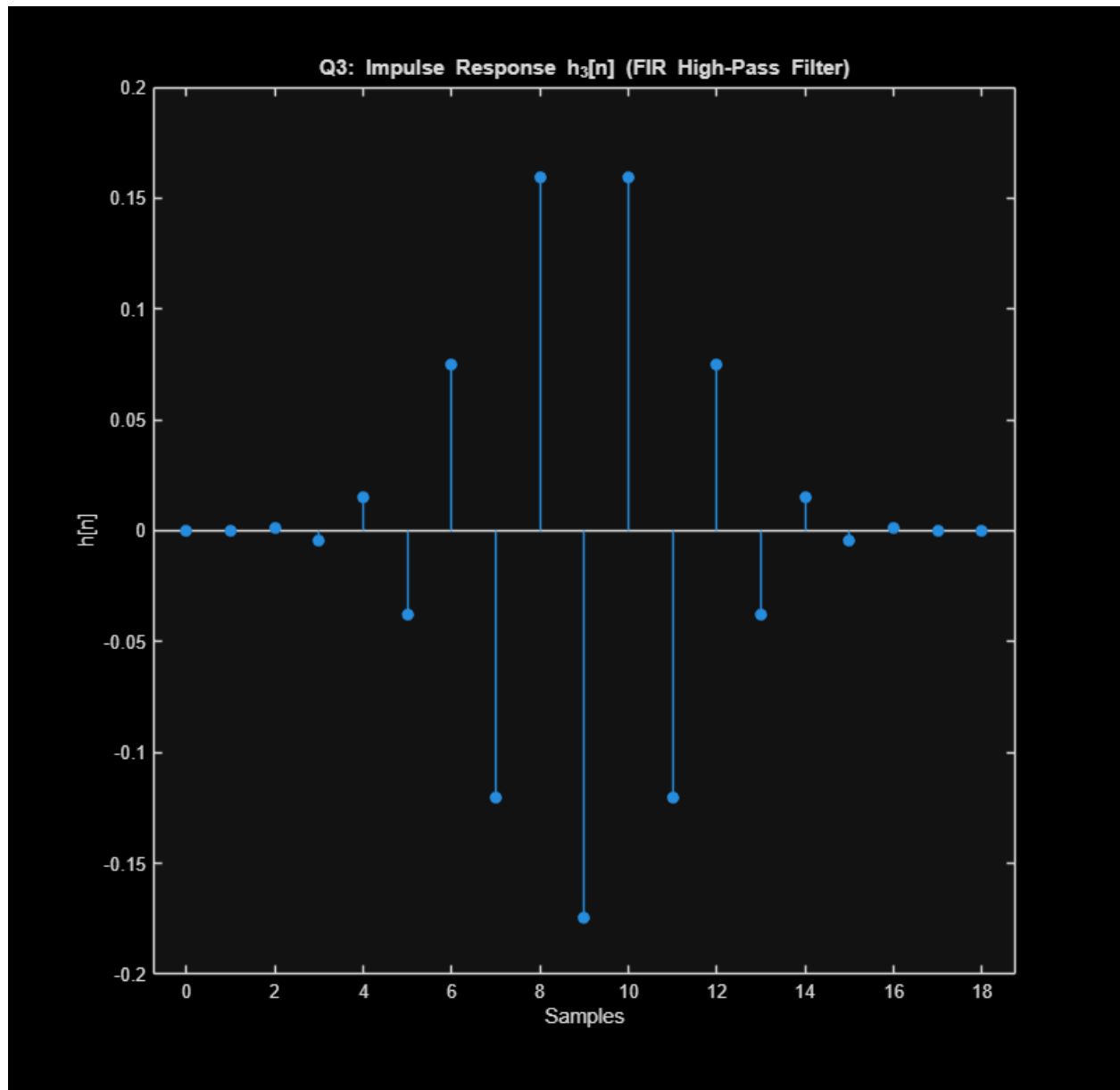
% -----
% 3(l)

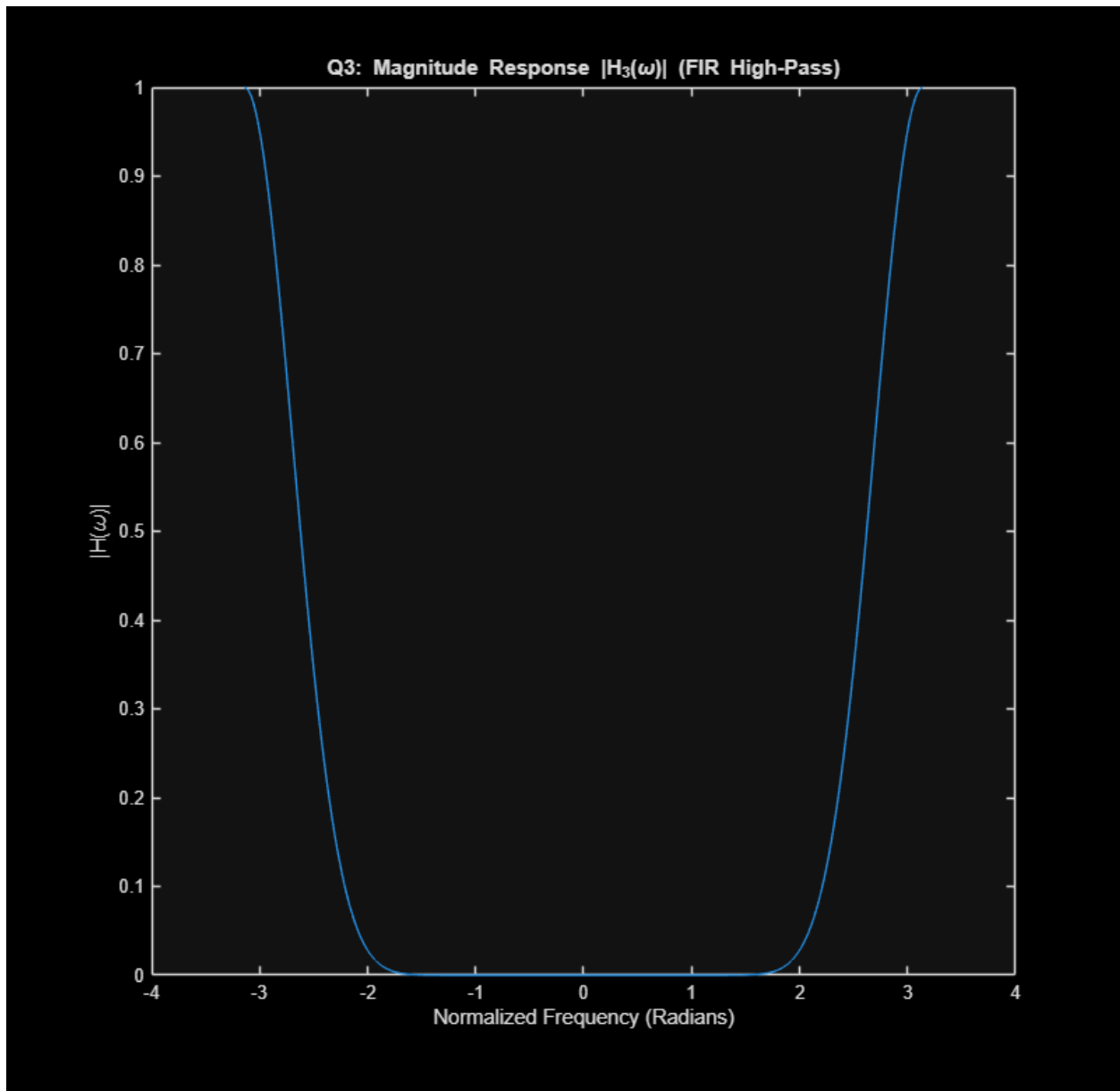
% Save (as q3.mat) your filter coefficients using MATLAB's save function.
% The filter coefficients
% should be in variables called b3 (numerator) and a3 (denominator).
% -----

save('q3.mat', 'b3', 'a3');
```







ALL FUNCTIONS SUPPORTING THIS CODE % % function pzplot(b,a)

PZPLOT(B,A) plots the pole-zero plot for the filter described by vectors A and B. The filter is a "Direct Form II Transposed" implementation of the standard difference equation:

```
a(1)*y(n) = b(1)*x(n) + b(2)*x(n-1) + ... + b(nb+1)*x(n-nb)
           - a(2)*y(n-1) - ... - a(na+1)*y(n-na)

% MODIFY THE POLYNOMIALS TO FIND THE ROOTS
b1 = zeros(max(length(a),length(b)),1); % Need to add zeros to get the
right roots
a1 = zeros(max(length(a),length(b)),1); % Need to add zeros to get the
```

```

right roots
    b1(1:length(b)) = b;      % New a with all values
    a1(1:length(a)) = a;      % New a with all values

    % FIND THE ROOTS OF EACH POLYNOMIAL AND PLOT THE LOCATIONS OF THE ROOTS
    h1 = plot(real(roots(a1)), imag(roots(a1)));
    hold on;
    h2 = plot(real(roots(b1)), imag(roots(b1)));
    hold off;

    % DRAW THE UNIT CIRCLE
    circle(0,0,1)

    % MAKE THE POLES AND ZEROS X's AND O's
    set(h1, 'LineStyle', 'none', 'Marker', 'x', 'MarkerFaceColor','none',
'linewidth', 1.5, 'markersize', 8);
    set(h2, 'LineStyle', 'none', 'Marker', 'o', 'MarkerFaceColor','none',
'linewidth', 1.5, 'markersize', 8);
    axis equal;

    % DRAW VERTICAL AND HORIZONTAL LINES
    xminmax = xlim();
    yminmax = ylim();
    line([xminmax(1) xminmax(2)], [0 0], 'linestyle', ':', 'linewidth', 0.5,
'color', [1 1 1]*.1)
    line([0 0], [yminmax(1) yminmax(2)], 'linestyle', ':', 'linewidth', 0.5,
'color', [1 1 1]*.1)

    % ADD LABELS AND TITLE
    xlabel('Real Part')
    ylabel('Imaginary Part')
    title('Pole-Zero Plot')

end

function circle(x,y,r)
% CIRCLE(X,Y,R)  draws a circle with horizontal center X, vertical center
% Y, and radius R.
%
% ANGLES TO DRAW
ang=0:0.01:2*pi;

% DEFINE LOCATIONS OF CIRCLE
xp=r*cos(ang);
yp=r*sin(ang);

% PLOT CIRCLE
hold on;
plot(x+xp,y+yp, ':', 'linewidth', 0.5, 'color', [1 1 1]*.1);
hold off;

end

```

```

function H = DTFT(x,w)
% DTFT(X,W)  compute the Discrete-time Fourier Transform of signal X
% across frequencies defined by W.

    H = zeros(length(w),1);
    for nn = 1:length(x)
        H = H + x(nn).*exp(-1j*w.'*(nn-1));
    end

end

function xs = shift(x, s)
% SHIFT(x, s) shifts signal x by s such that the output can be defined by
% xs[n] = x[n - s]

    % INITIALIZE THE OUTPUT
    xs = zeros(length(x), 1);

    for n = 1:length(x)
        % CHECK IF THE SHIFT IS OUT OF BOUNDS FOR THIS SAMPLE
        if n-s > 0 && n-s < length(x)
            % SHIFT DATA
            xs(n) = x(n-s);
        end
    end

end

function [b,a] = pz2ba(p,z)
% PZ2BA(P,Z)  Converts poles P and zeros Z to filter coefficients
%              B and A
%
% Filter coefficients are defined by:
%      a(1)*y(n) = b(1)*x(n) + b(2)*x(n-1) + ... + b(nb+1)*x(n-nb)
%                  - a(2)*y(n-1) - ... - a(na+1)*y(n-na)
%
% CONVERT ROOTS (POLES AND ZEROS) INTO POLYNOMIALS
b = poly(z);
a = poly(p);

end

function [p,z] = ba2pz(b,a)
% BA2PZ(B,A)  Converts filter coefficients B and A into poles P and zeros Z
%
% Filter coefficients are defined by:

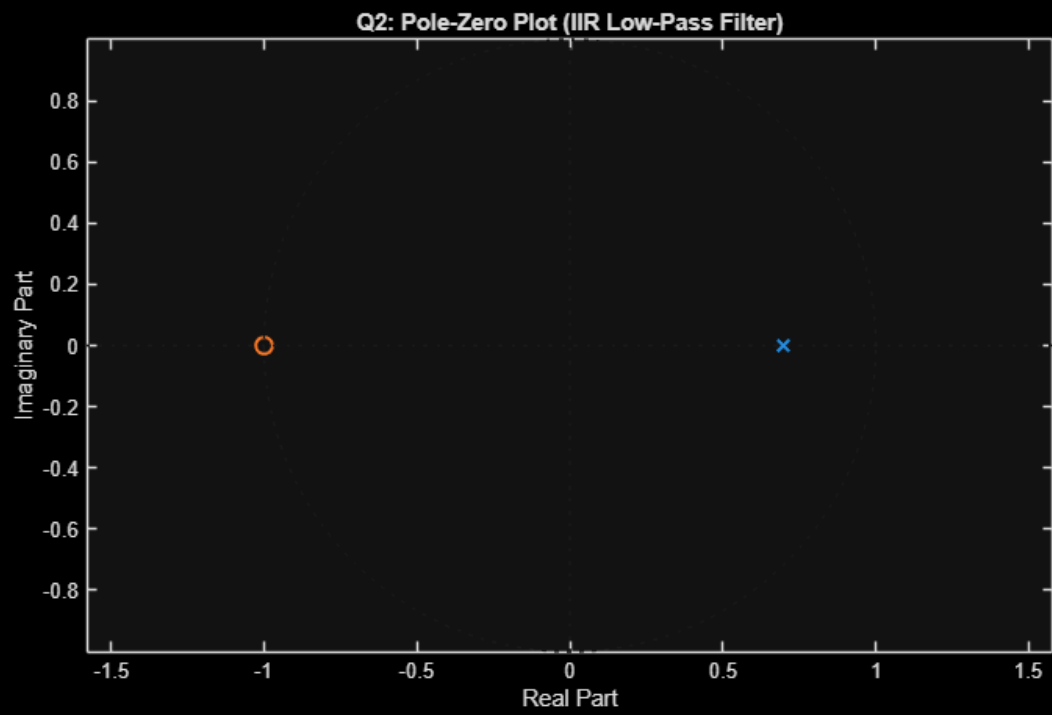
```

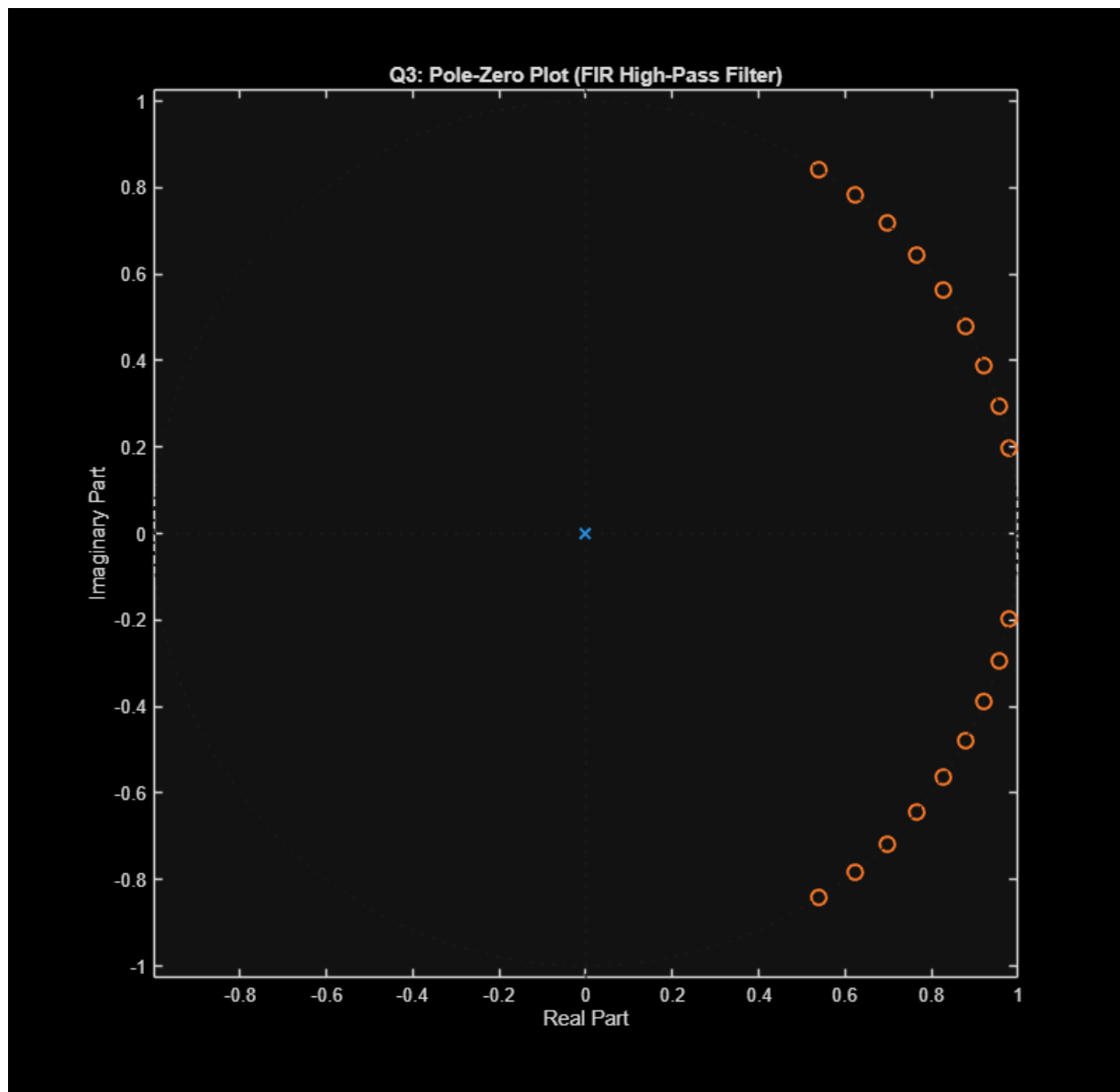
```
%      a(1)*y(n) = b(1)*x(n) + b(2)*x(n-1) + ... + b(nb+1)*x(n-nb)
%
%      - a(2)*y(n-1) - ... - a(na+1)*y(n-na)
%

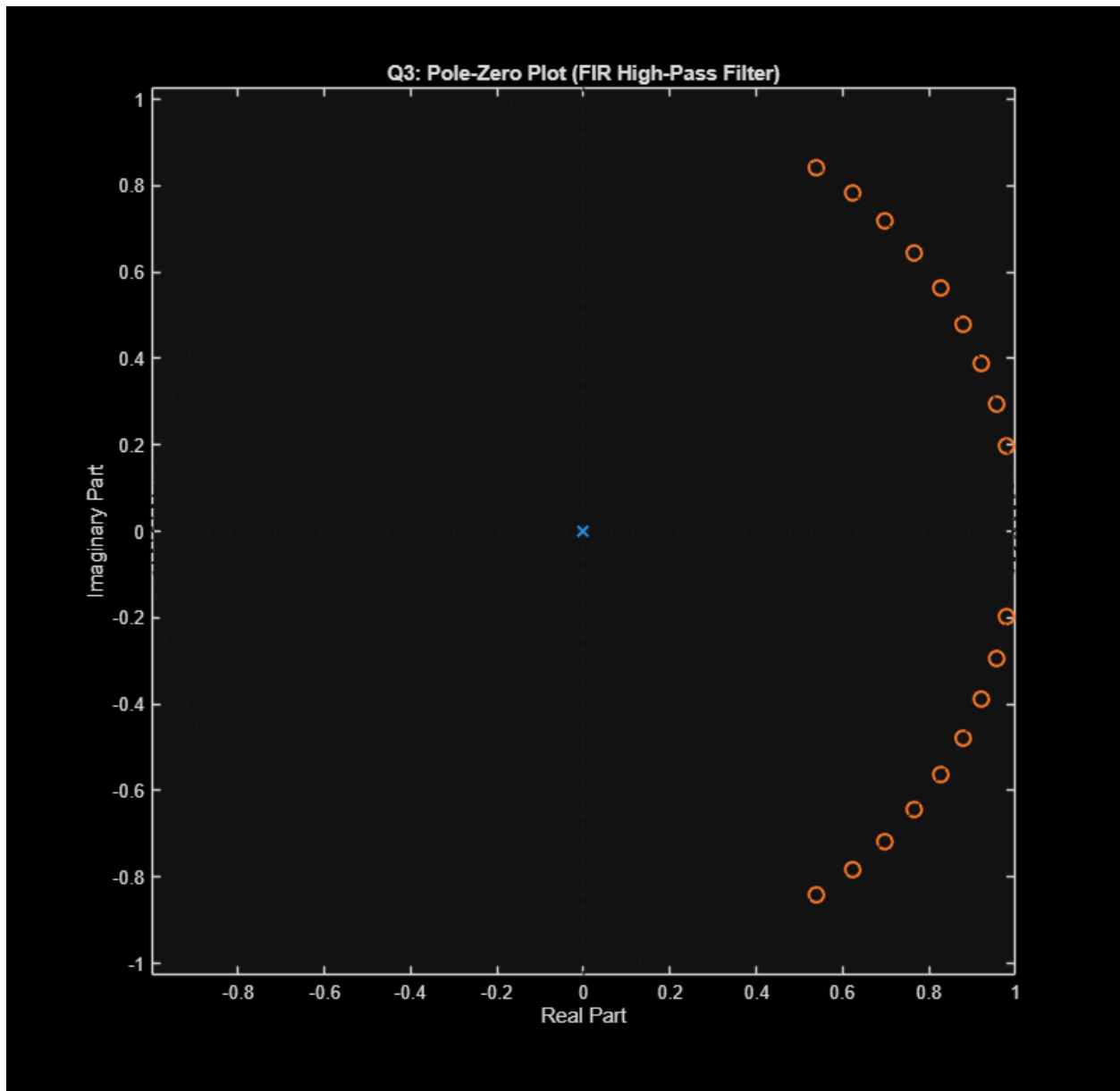
% MODIFY THE POLYNOMIALS TO FIND THE ROOTS
b1 = zeros(max(length(a),length(b)),1); % Need to add zeros to get the
right roots
a1 = zeros(max(length(a),length(b)),1); % Need to add zeros to get the
right roots
b1(1:length(b)) = b;      % New a with all values
a1(1:length(a)) = a;      % New a with all values

% FIND THE ROOTS OF EACH POLYNOMIAL AND PLOT THE LOCATIONS OF THE ROOTS
p = real(roots(a1))+1j*imag(roots(a1));
z = real(roots(b1))+1j*imag(roots(b1));

end
```





Published with MATLAB® R2025b